FinSight — An Intelligent Personal Finance Analytics Platform with MLOps-Driven Insights and Cloud-Native DevOps Automation

Project description

FinSight is an intelligent personal finance analytics platform that enables users to register, securely authenticate, and upload their bank transaction history for automated parsing into a canonical data schema. The platform delivers ML-powered spending insights — including categorization, anomaly detection, spend forecasting, and a financial stress score — through a personalized dashboard, budget management tools, and a conversational AI assistant grounded in the user's own financial data. Built on a microservices architecture and deployed through a full DevOps pipeline (Jenkins, Docker, Kubernetes, Ansible, ELK, Vault), FinSight operationalizes MLOps best practices including model versioning, drift detection, and automated retraining.

Functional requirements

User registration & profile

- User must be able to register with name, email, password, and basic financial profile (monthly income, city, age group)
- User must be able to sign in via Google OAuth as an alternative to email/password
- System must validate email uniqueness and enforce password strength rules at registration
- User must be able to update their profile, change password, and delete their account
- Admin must be able to view all registered users, deactivate accounts, and manage platform-wide settings

Authentication & authorization

- System must issue JWT tokens on successful login with configurable expiry
- Google OAuth flow must create a new account on first login or link to an existing account if the email matches
- All protected routes must validate the JWT token via the API gateway before forwarding requests
- Role-based access control must distinguish between the `user` and `admin` roles with separate permission sets
- Sessions must be invalidated on logout and token refresh must be supported

Bank statement ingestion & parsing

- User must be able to upload a bank statement in CSV, XLS, or PDF format
- An intelligent parsing service must extract transactions regardless of bank-specific column formats (date, description, amount, debit/credit, balance)
- Parsed transactions must be normalized into a canonical schema: `transaction_id`, `user_id`, `date`, `description`, `amount`, `type` (debit/credit), `merchant`, `category`, `source_bank`, `raw_text`
- User must be able to review parsed transactions before confirming ingestion
- User must be able to manually correct or override any parsed field
- System must deduplicate transactions on re-upload using date + amount + description hash
- Ingestion events must be published to Kafka for downstream ML services to consume asynchronously

Transaction management

- User must be able to view all transactions in a filterable, sortable table (by date, category, amount, merchant)
- User must be able to manually add, edit, or delete individual transactions
- User must be able to split a transaction across multiple categories with custom amounts
- User must be able to tag transactions with custom labels
- User must be able to set up rules to auto-categorize future transactions by merchant name or keyword
- System must automatically re-categorize affected transactions when a rule is created or modified

Spending categorization

- ML categorization service must classify each transaction into a standard category set: food & dining, transport, shopping, utilities, healthcare, entertainment, education, rent, income, transfer, and other
- User must be able to reassign a transaction to a different category
- User-corrected categories must feed back into the categorization model as labeled training data
- System must display a category breakdown as a percentage and absolute amount for any selected time period

Spending dashboard & analytics

- User must see a personal dashboard showing total spend, total income, and net savings for the current month
- Dashboard must display a category-wise spending breakdown with trend comparison against the previous month
- User must be able to select custom date ranges for all analytics views
- Dashboard must show top merchants by spend for the selected period
- System must display a month-on-month spending trend chart for the last 12 months
- User must be able to export their spending report as a PDF or CSV for any date range

Budget management

- User must be able to set a monthly budget limit per spending category
- System must track actual spend against each budget in real time and show a progress indicator
- System must send an in-app notification when spend reaches 80% of a budget limit
- System must send an in-app notification when a budget limit is exceeded
- User must be able to carry forward unspent budget to the next month or reset it
- Admin must be able to view aggregate budget adherence statistics across all users

ML-powered anomaly detection

- Anomaly detection service must build a per-user baseline model from at least 3 months of transaction history
- Service must score every new transaction against the user's personal baseline at ingestion time
- Transactions with an anomaly score above a configurable threshold must trigger a real-time alert to the user
- Alert must include a plain-language explanation of why the transaction was flagged (unusual merchant, unusual time, unusual amount relative to baseline)
- User must be able to mark an alert as "expected" or "suspicious" — both responses must be stored as labeled feedback
- Feedback must be used to retrain the anomaly model for that user in the next scheduled retraining cycle

Spending forecaster

- Forecasting service must predict next month's total spend per category based on historical transaction patterns
- Forecast must be displayed on the dashboard as "predicted vs budgeted" before the month begins
- Service must update the forecast at mid-month using actual spend to date plus projected remainder
- User must be able to see the model's confidence interval alongside the forecast
- Forecast accuracy per user must be tracked over time and surfaced in the admin dashboard

Financial stress score

- Stress scoring service must compute a composite score (0–100) monthly from: spend-to-income ratio, savings rate, EMI/recurring burden, discretionary spend growth, and budget adherence rate
- Score must be displayed as a trend line on the user's dashboard showing the last 6 months
- System must generate a plain-language explanation of the key drivers behind the current month's score
- Score changes of more than 10 points month-on-month must trigger a proactive alert to the user

What-if simulator

- User must be able to open a simulator panel and adjust input sliders: reduce category spend by X%, add a new recurring expense of ₹Y, or remove an existing subscription
- Simulator must call the forecasting and stress scoring services in real time and display the projected impact on next month's balance and stress score
- Simulator results must not affect the user's actual transaction data or live model state

AI chat assistant

- Chat service must accept free-text financial questions from the user
- Service must route each query to the appropriate backend microservices (transaction service, forecasting service, anomaly service, stress scorer) to retrieve real data
- Responses must be grounded in the user's actual financial data, not generic answers
- Chat must support at least the following query intents: spending summary for a period, category comparison, anomaly explanation, forecast inquiry, budget status, and what-if scenario
- Chat history must be persisted per user session and accessible in the UI

- User must be able to rate each chat response as helpful or not helpful — ratings must be logged for model improvement

Notifications & alerts

- System must deliver real-time in-app notifications for: anomalous transaction detected, budget limit approaching, budget limit exceeded, financial stress score change, and forecast update available
- User must be able to configure which notification types they want to receive
- All notifications must be stored and viewable in a notifications inbox
- Admin must be able to broadcast platform-wide announcements to all users

Admin panel

- Admin must be able to view platform-wide metrics: total users, total transactions ingested, average stress score distribution, and ML model performance stats
- Admin must be able to trigger a manual model retraining run for any service
- Admin must be able to configure ML model thresholds (anomaly sensitivity, stress score weights) without redeployment
- Admin must be able to view all ingestion jobs and their parse success/failure rates
- Admin must be able to view the ELK Kibana dashboard embedded within the admin panel

---

Non-functional requirements

Performance

- API gateway must respond to all requests within 500ms at the 95th percentile under normal load
- ML inference endpoints must return a score within 200ms for single-transaction anomaly detection
- Bank statement parsing service must process a 1,000-row CSV within 10 seconds
- Dashboard must load within 2 seconds for up to 24 months of transaction history

Scalability

- ML inference service must be configured with Kubernetes HPA to scale from 2 to 8 replicas based on CPU utilization exceeding 70%
- Kafka must handle at least 500 transaction ingestion events per second without message loss

- All microservices must be stateless to allow horizontal scaling without session affinity

Availability & reliability

- Platform must target 99.5% uptime for all user-facing services
- Kubernetes rolling update strategy must ensure zero downtime during deployments (`maxUnavailable: 0`, `maxSurge: 1`)
- All services must implement health check endpoints (`/health`) polled by Kubernetes liveness and readiness probes
- Failed Kafka messages must be routed to a dead-letter queue and retried up to 3 times before alerting

Security

- All secrets (database credentials, OAuth client secrets, ML model registry tokens, Kafka credentials) must be stored in HashiCorp Vault and injected at runtime — no plaintext secrets in code or config files
- All inter-service communication must use TLS
- User passwords must be hashed with bcrypt (minimum cost factor 12)
- JWT tokens must be signed with RS256 and rotated every 24 hours
- Bank statement files must be encrypted at rest and deleted from the ingestion service after successful parsing
- All API endpoints must enforce rate limiting: 100 requests per minute per authenticated user
- OWASP Top 10 vulnerabilities must be addressed in the API gateway layer

Observability

- Every microservice must emit structured JSON logs to Logstash via the ELK pipeline
- Kibana must have pre-built dashboards for: transaction ingestion rate, ML inference latency, anomaly alert volume, and error rate per service
- Prometheus must scrape metrics from all services; Grafana dashboards must show CPU, memory, and request throughput per pod
- All distributed traces must include a correlation ID propagated across service calls for end-to-end request tracing

DevOps & deployment

- Every push to the main branch must trigger the Jenkins CI/CD pipeline automatically via GitHub webhook

- Jenkins pipeline must include stages: build, unit test, integration test, Docker image build, push to Docker Hub, Ansible deploy, Kubernetes apply, smoke test
- Ansible must use at minimum 4 roles: `common`, `kafka`, `kubernetes`, `monitoring`
- All Kubernetes manifests must be version-controlled and applied idempotently
- MLflow must version every trained model; only models passing evaluation thresholds may be promoted to production

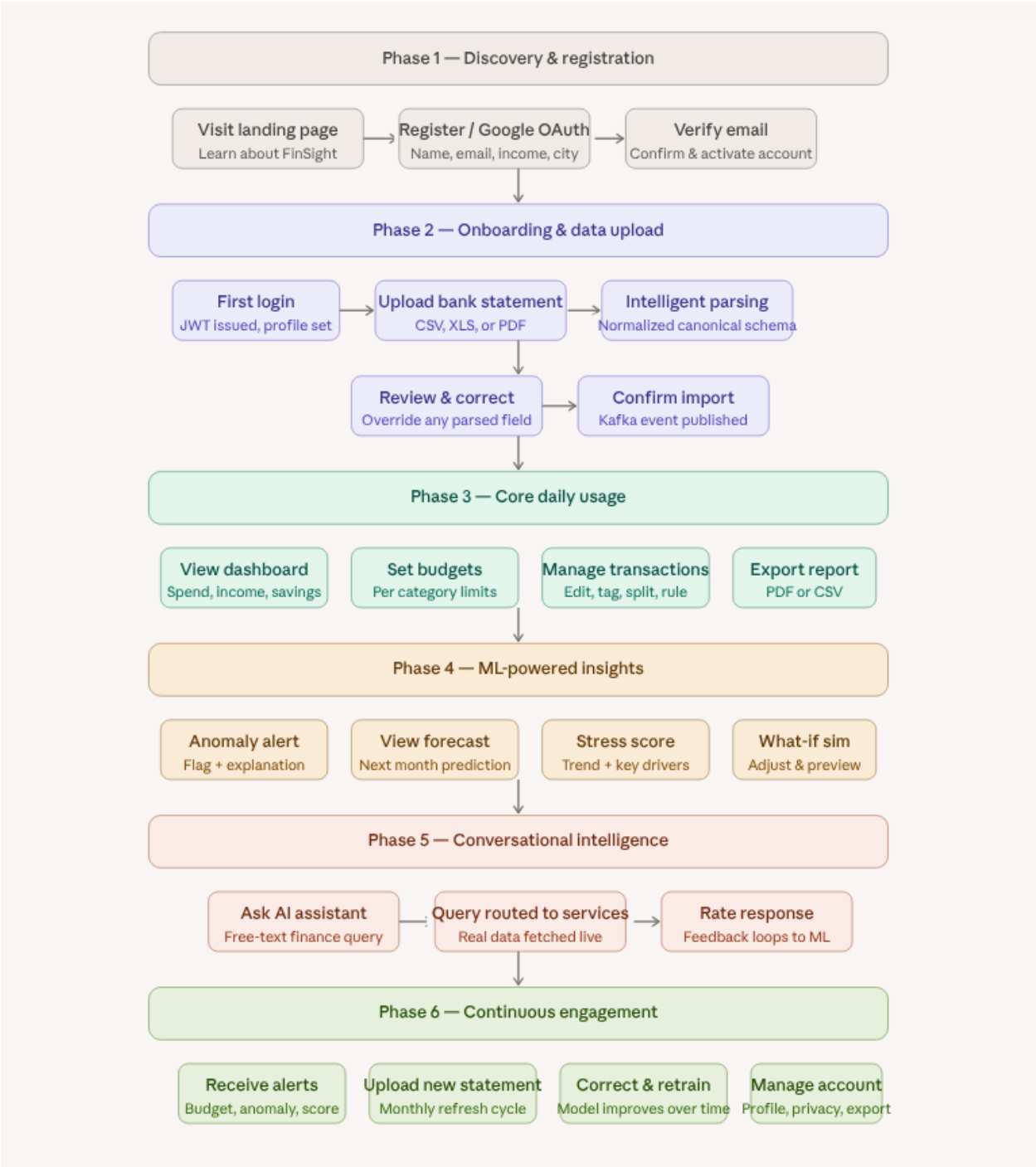
Data & compliance

- All user financial data must be stored in an isolated PostgreSQL schema per user to prevent cross-user data leakage
- Data retention policy must allow users to request full deletion of their data (right to erasure)
- Parsed transaction data must never be shared with third parties
- Audit logs of all admin actions must be immutable and retained for 90 days

---

User journey

Here is the complete end-to-end journey from first visit to daily engaged use:



Every phase maps directly to requirements above — Phase 1 covers registration, Phase 2 covers ingestion, Phase 3 covers core features, Phase 4 covers ML services, Phase 5 covers the chat assistant, and Phase 6 covers the continuous feedback loop that drives the MLOps retraining story.

Features

Core (expected, but done well)

Spending categorization Auto-categorize transactions (food, transport, rent, entertainment) using an NLP classifier trained on merchant names and descriptions. Most apps do rule-based categorization — ML-based is already a step ahead.

Monthly budget tracker Users set budgets per category; a progress bar fills in real time as transactions come in; alerts fire at 80% and 100% of budget. Standard but necessary.

Recurring payment detector Automatically identifies subscriptions and EMIs from transaction patterns, groups them, and shows total recurring commitment per month.

Intermediate (good differentiation)

Peer benchmarking (anonymized) "You spend 34% more on dining than people in your income bracket and city." Aggregate anonymized spending patterns across your user base and let users compare themselves to peers — segmented by city, age group, or income range. No other feature makes users reflect on behavior as effectively.

Merchant intelligence Every merchant gets a profile — average spend per visit, visit frequency, price trend over time. Users see "you visit Swiggy 12 times a month, spending ₹450 on average per order — that's ₹5,400/month on food delivery alone." Simple calculation, surprising insight.

Spend velocity alerts Not just "you exceeded budget" but "you have spent ₹8,000 in the last 3 days, which puts you on track to overspend by ₹12,000 this month." A linear projection model runs continuously and fires an early warning before the damage is done.

Bill split tracker Users tag transactions as "split with friends," enter who owes what, and the platform tracks outstanding balances. Settlement suggestions fire when a friend pays back. Social finance layer on top of personal finance.

Advanced ML features (strong innovation marks)

Personalized anomaly detection (your core ML service) Instead of a global fraud rule ("transactions over ₹10,000 are suspicious"), the LSTM autoencoder learns each individual user's spending DNA — their usual merchants, times, amounts, and frequencies. A ₹500 transaction at a petrol pump at 11pm is normal for a sales professional and anomalous for a

student who has never visited one. The model flags deviations from the user's own baseline, not a population average. This is the key architectural differentiator.

Future spend forecasting Given the last 3 months of transactions, predict next month's spending per category using a time-series model. Show users a "predicted vs budget" view before the month even starts. "Based on your patterns, you are likely to spend ₹6,200 on dining next month — ₹1,200 over your budget." Gives users agency to adjust behavior proactively.

Financial stress score A composite ML-derived score (0–100) that combines: spend-to-income ratio, savings rate trend, EMI burden, discretionary spend growth, and emergency fund adequacy. Updated monthly. Users see a trend line — is their financial health improving or deteriorating? Analysts see aggregate stress distribution across the user base. No consumer app in India currently does this well.

Impulse purchase detector Classify transactions by time of day, day of week, merchant category, and amount to identify likely impulse purchases (late-night food delivery, weekend shopping spikes, flash sale purchases). Show users their "impulse spend" as a separate category with a monthly total. Behavioral nudges fire after a detected impulse: "You have made 3 late-night food orders this week totalling ₹1,800."

Goal-based savings optimizer Users set a financial goal (vacation, laptop, emergency fund) with a target amount and date. The optimizer analyzes spending patterns, identifies cuttable categories, and generates a personalized savings plan: "If you reduce dining by 30% and cancel 2 streaming subscriptions, you will reach your goal 6 weeks earlier." Recalculates dynamically as actual spending comes in.

Genuinely novel features (innovation marks territory)

Spending mood correlation Users optionally log their mood daily (a 10-second check-in — happy, stressed, bored, sad). Over time, an ML model discovers correlations: "You tend to spend 60% more on food delivery on days you rate as stressful." Show this as a mood-spend heatmap. No mainstream app does this. The insight alone changes behavior.

What-if financial simulator An interactive simulator where users drag sliders — "what if I reduce dining by ₹2,000/month?" or "what if I take an EMI of ₹5,000?" — and see the projected impact on their financial stress score, savings goal timeline, and month-end balance in real time. The ML forecasting model runs on every slider move. Makes the AI feel like a live financial advisor, not a static report.

Contextual spending explainer (chat interface) Users ask natural language questions and the chat service queries the backend microservices to answer:

- "Why did I overspend in March?" → anomaly service + category breakdown
- "Am I spending more on food than last year?" → trend analysis service

- "Can I afford a ₹3,000 EMI right now?" → financial stress scorer + forecaster
- "Which subscriptions am I not using?" → recurring payment detector + frequency analyzer

Each answer is generated by composing real data from multiple services — not a canned response. This makes the chat genuinely intelligent rather than FAQ-style.

Social savings challenges Users invite friends to a shared savings challenge — "we all reduce dining spend by 20% this month." A leaderboard shows progress. Peer accountability is one of the strongest behavioral nudges in personal finance research. Gamification layer on top of the analytics platform.

Merchant category price inflation tracker Track how the user's average spend per visit at recurring merchants changes over time. "Your average grocery bill at DMart has increased 18% over the last 6 months — likely price inflation." Contextualizes spending changes so users don't blame themselves for increases driven by external factors.

Technology stack — FinSight

Frontend

- React 18 with TypeScript — component framework
- Vite — build tool and dev server (faster than CRA)
- Tailwind CSS — utility-first styling
- Recharts — spending charts, trend lines, category breakdowns
- React Query (TanStack Query) — server state management and API caching
- Zustand — lightweight client state management (auth, user session)
- React Router v6 — client-side routing
- Axios — HTTP client with interceptors for JWT attachment
- Google OAuth via `@react-oauth/google` — OAuth flow on the frontend

Backend microservices (Java Spring Boot)

All services use Java 21 + Spring Boot 3.x. Each is an independent deployable unit with its own database and Dockerfile.

Service	Responsibility	Key Spring dependencies
`api-gateway-service`	Auth, routing, rate limiting	Spring Cloud Gateway, Spring Security, JWT (jjwt)
`user-service`	Registration, login, OAuth, profile	Spring Security, OAuth2 Client, Spring Data JPA
`ingestion-service`	File upload, parse orchestration, Kafka publish	Spring Kafka, Apache POI (XLS), PDFBox (PDF), OpenCSV
`transaction-service`	CRUD, rules engine, deduplication	Spring Data JPA, Spring Kafka consumer
`budget-service`	Budget limits, tracking, threshold alerts	Spring Data JPA, Spring Kafka
`notification-service`	In-app alerts, inbox management	Spring Kafka consumer, WebSocket (STOMP)
`admin-service`	User management, platform metrics, config	Spring Data JPA, Spring Security
`chat-service`	Query intake, route to ML services, response assembly	Spring WebFlux (reactive), WebClient

---

ML microservices (Python)

All ML services use Python 3.11 + FastAPI. Kept separate from Spring Boot intentionally — Python owns the data science ecosystem and these services are called by Spring Boot services via REST.

Service	Responsibility	Key libraries
`parsing-ml-service`	Intelligent bank statement parsing, column detection	pandas, pdfplumber, openpyxl, spaCy
`categorization-service`	ML transaction categorization	scikit-learn, XGBoost, FastAPI
`anomaly-detection-service`	Per-user LSTM anomaly scorer	PyTorch, numpy, FastAPI
`forecasting-service`	Category-wise spend forecasting	PyTorch (LSTM), statsmodels, FastAPI
`stress-score-service`	Composite financial stress scorer	scikit-learn, pandas, FastAPI
`simulation-service`	What-if scenario computation	pandas, FastAPI (calls forecasting + stress scorer)
`model-training-service`	Scheduled retraining, drift detection, MLflow tracking	MLflow, PyTorch, scikit-learn, scipy (KL divergence)

Databases

Store	Used by	Why
PostgreSQL 16	user-service, transaction-service, budget-service, admin-service	Relational — structured financial data, ACID compliance
MongoDB	ingestion-service	Raw uploaded file metadata, flexible parsing job documents
Redis 7	api-gateway (rate limiting, token cache), chat-service (session), simulation-service (cache)	Sub-millisecond reads, ephemeral data

Each Spring Boot service that owns a PostgreSQL schema uses Flyway for database migrations.

Messaging

- Apache Kafka 3.x — event streaming between ingestion, transaction, ML, and notification services
- Topics: `transactions.ingested`, `transactions.categorized`, `anomaly.alerts`, `budget.alerts`, `model.retrain.trigger`
- Kafka is consumed by Spring Boot services via `spring-kafka` and by Python ML services via `kafka-python`

ML lifecycle

- MLflow — experiment tracking, model versioning, model registry
- Model artifacts stored in MLflow with PostgreSQL as the tracking backend
- Drift detection: scipy KL divergence computed in `model-training-service`
- Retraining triggered either by drift threshold breach or manual admin action via `admin-service` → Kafka → `model-training-service`

DevOps & infrastructure

Tool	Purpose
Git + GitHub	Version control, webhook trigger
Jenkins	CI/CD pipeline, GitHub webhook receiver
Docker + Docker Compose	Containerization, local dev environment
Docker Hub	Container image registry
Kubernetes (K8s)	Production orchestration
Kubernetes HPA	Auto-scaling `anomaly-detection-service` and `categorization-service` on CPU
Ansible	Configuration management — 4 roles: `common`, `kafka`, `kubernetes`, `monitoring`
HashiCorp Vault	Secrets management — DB passwords, OAuth secrets, Kafka credentials, MLflow tokens

---

Observability

Tool	Purpose

Elasticsearch	Log storage and indexing
Logstash	Log aggregation from all services
Kibana	Dashboards — ingestion rate, inference latency, error rate
Prometheus	Metrics scraping from all services
Grafana	Infrastructure dashboards — CPU, memory, request throughput per pod
Spring Boot Actuator	Exposes `/health`, `/metrics` endpoints for K8s probes and Prometheus
Micrometer	Bridges Spring Boot metrics to Prometheus format

Inter-service communication

- Synchronous (REST): React → API Gateway → Spring Boot services → Python ML services via `WebClient` (reactive, non-blocking)
- Asynchronous (event-driven): Spring Boot → Kafka → Spring Boot / Python consumers
- WebSocket (STOMP over SockJS): `notification-service` → React frontend for real-time alerts

---

Authentication flow

- Standard login: `user-service` validates credentials → issues JWT (RS256) → stored in httpOnly cookie on frontend

- Google OAuth: React uses `@react-oauth/google` → sends `id_token` to `user-service` → validated via Google's JWKS endpoint → JWT issued
- All subsequent requests: JWT validated at `api-gateway-service` via Spring Security filter before routing

---

Local development setup

- Docker Compose spins up: all Spring Boot services, all Python ML services, PostgreSQL, MongoDB, Redis, Kafka + Zookeeper, MLflow, ELK stack
- Each Spring Boot service has an `application-local.yml` profile that points to local Docker network addresses
- Python services use `.env` files locally; Vault replaces these in production
- A `demo_data` seed script populates 6 months of synthetic transactions for a test user

---

The split between Spring Boot and Python is deliberate and clean. Spring Boot owns everything that is user-facing and transactional — auth, CRUD, routing, business logic, notifications. Python owns everything that is data-science — model inference, training, parsing intelligence, drift detection. The two worlds communicate only via REST (Spring Boot calls Python FastAPI endpoints) and Kafka (Spring Boot publishes events, Python consumers process them). This separation means you can develop and deploy each layer independently, which is exactly what your microservices architecture requires.